

Lab 1: Getting Ready for Class

due Tuesday 06/23 at 11:30am (EST)

Adapted from materials designed for Harvey Mudd's CS 5 course (Fall 2015)

Introduction

Before we can really get started with learning to program, there are a couple of important first steps. Namely, you'll need to install Python and VSCode (the text editor we'll be using to write and edit code this term) and set up something called a *virtual environment* which will allow you to manage the additional packages you'll need for class.

Creating a Course Folder

For practicality's sake, we'll store all of the work you do for this course – including labs, assignments, and GRQs – in a single folder. You can choose to place this folder wherever you want; I like to store things in `Documents`.

Open Finder on Mac or File Explorer on Windows and then navigate to the location of your choice. Then, create a new folder and name it `info1100`.

Open the `info1100` folder. Now, create three additional subfolders within `info1100` (which we'd call a "parent" folder): `labs`, `grqs`, and `assignments`.

Installing VSCode

For this class, we'll be writing and running code using a text editor called VSCode. This is free software that works with all operating systems. It absolutely isn't the only way you can write or edit code, but for the sake of this course it'll help us be certain that everyone is using the same software and set up.

The [download page for VSCode](#) should be pretty self-explanatory; it's like downloading any other app for your computer! Try to set it up on your own and reach out if you hit any problems.

After you download the app, I recommend that you start it up and choose to *Keep in Dock* (Mac) or *Pin to Taskbar* (Windows). It'll make your life easier if you can easily access it for the course of this term.

Using VSCode

Let's now take a stab at using VSCode as a text editor!

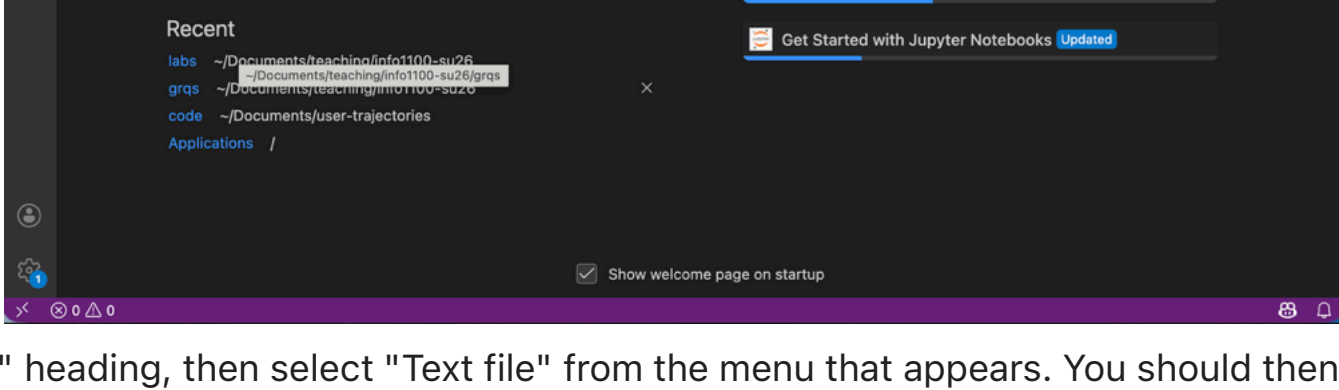
Text editors are different from word processors. Some differences are...

- Google Docs and Microsoft Word and other word processors format things beautifully
- Word processors don't allow you to see the actual contents of the file, like the special characters that determine document formatting
- All programming languages use **raw text** (strings of letters), plain-text editors are what we want to use for programming!

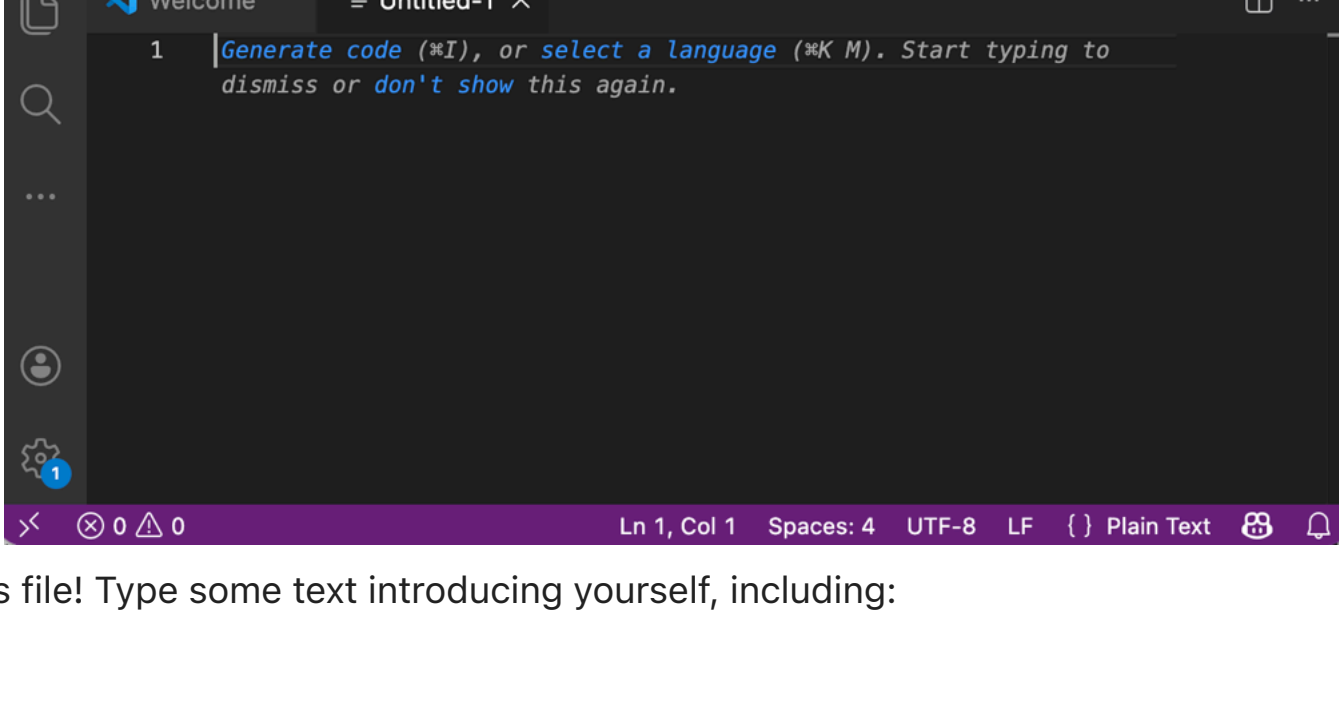
To get started using VSCode, first open the app – it should either be in the dock or taskbar, if you pinned it there, or in your Applications folder.

It'll start by asking you some welcome questions. Feel free to answer them as you like; color themes can be fun to play around with, so optimize to whatever you'd like, but I'd recommend skipping the "Sync to other devices" option. It'll be helpful later on if you can install the support for the Python language, which should be one of the first options, although you can do this later on if you'd prefer. It may ask you if you want to make a VSCode account; no need to do so if you don't want to!

Eventually, you will likely hit a Welcome page like the one depicted below:



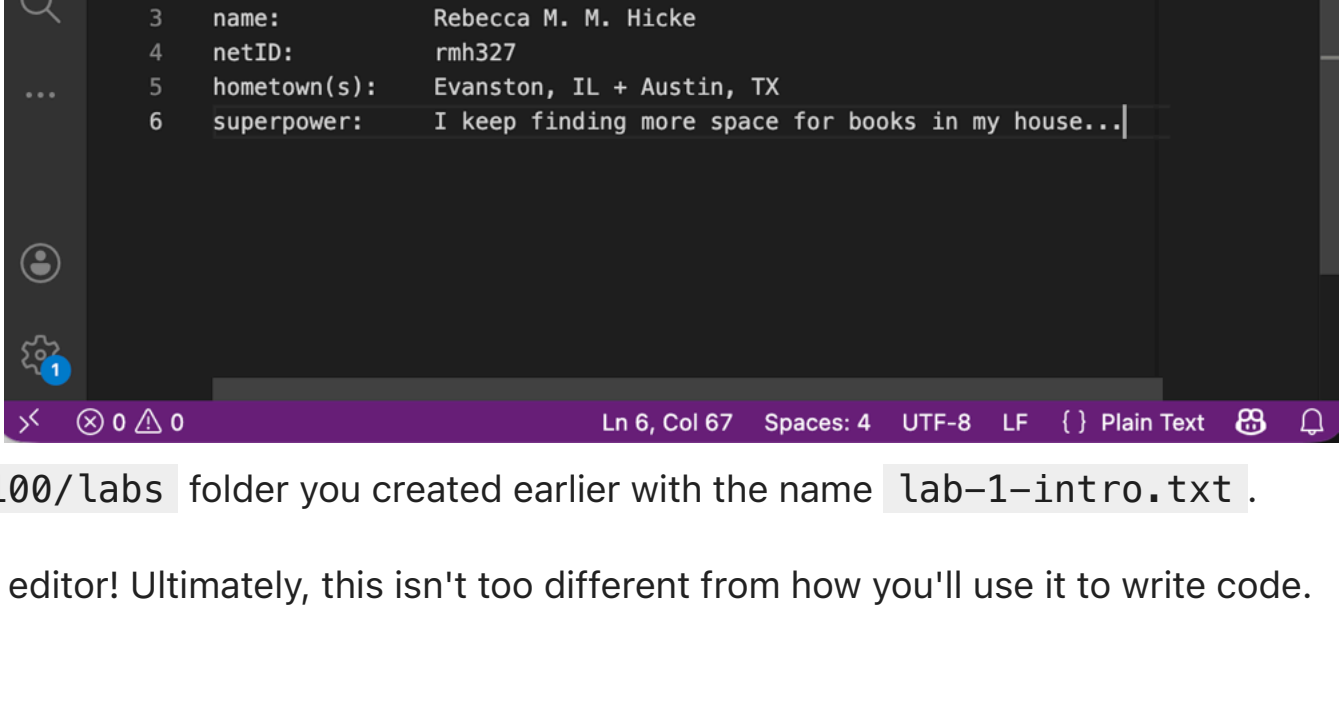
Click "New File..." under the "Start" heading, then select "Text file" from the menu that appears. You should then see something like this window:



Now, let's add some content to this file! Type some text introducing yourself, including:

- name
- netID
- hometown(s)
- your secret superpower

My version of this would look like:



Then, save your file to the `info1100/labs` folder you created earlier with the name `lab-1-intro.txt`.

You can now use VSCode as a text editor! Ultimately, this isn't too different from how you'll use it to write code.

Installing Python

We want to download the most recent version of Python. Almost all code today is written in Python 3, although an older version of Python called Python 2 still exists. These can differ in significant ways, so make sure you're downloading the right one!

To download Python for Mac, [click this link](#).

To download Python for Windows, [click this link](#).

If you use another operating system or just want to see the general download site for Python, look [here](#).

There are three ways we'll be running Python code in this class: directly from the Terminal, from a `.py` file, and in an interactive notebook. Let's test your Python installation to make sure it's working and introduce each way of running Python code.

Introducing the Terminal

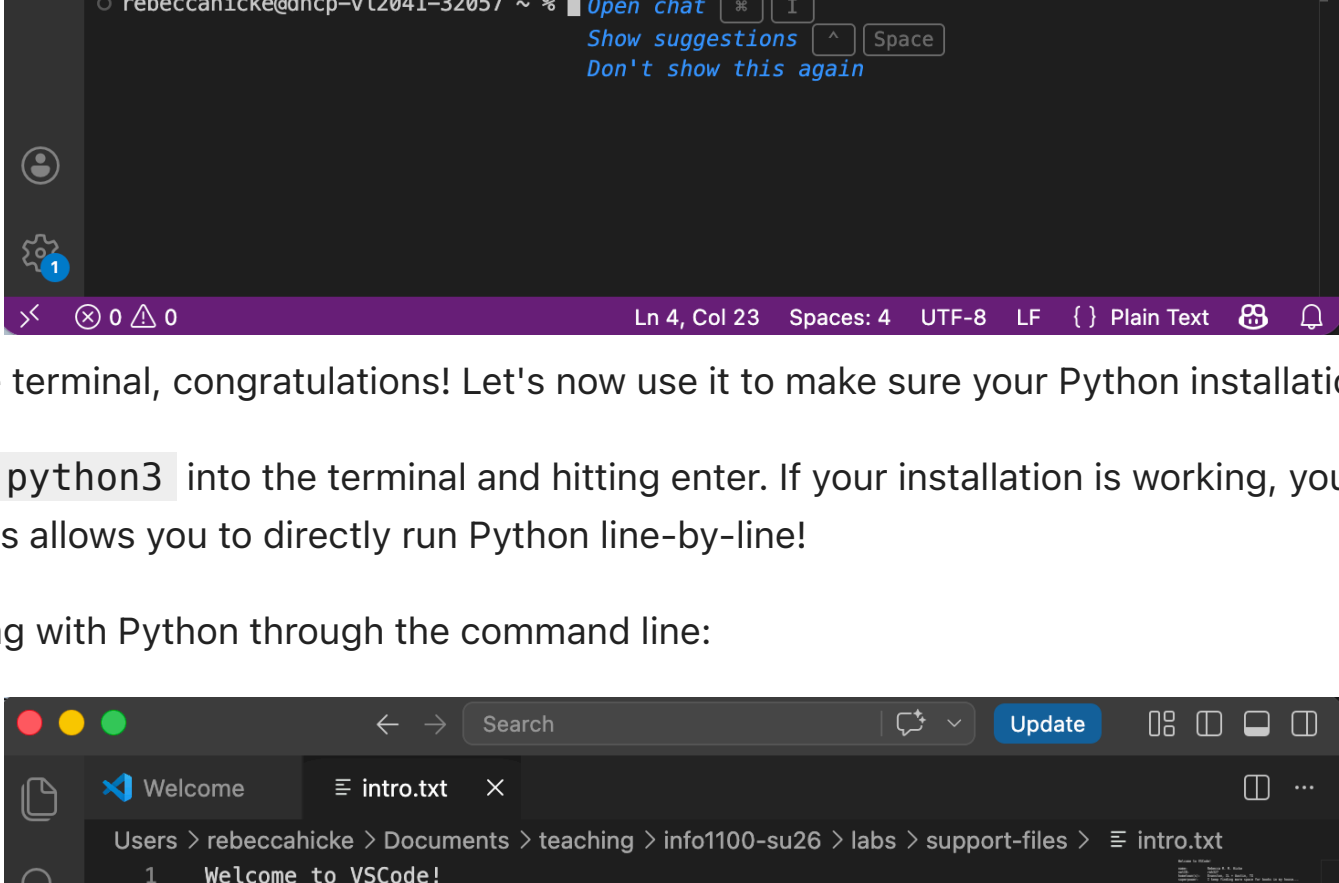
Most of our interactions with computers are through windows provided by the OS (or operating system). Three of the most common operating systems are Windows, Mac OS, and Linux – you're probably using one of the three right now!

The OS provides a *windowing system* that lets you use a mouse and interact with the computer in an intuitive way. Using an operating system to navigate your machine makes most day-to-day interactions with your computer much simpler!

However, the graphical interface the operating system hides some functionality that it'll be useful for us to have when we're working with code. The terminal uses textual commands, known as the **command line**, to handle files and run commands on your machine.

There are two ways we'll learn to access the terminal. The first is directly through the terminal. Go to the `lab-1-intro.txt` file you just created in VSCode, click on the `View` menu, and then select `Terminal`.

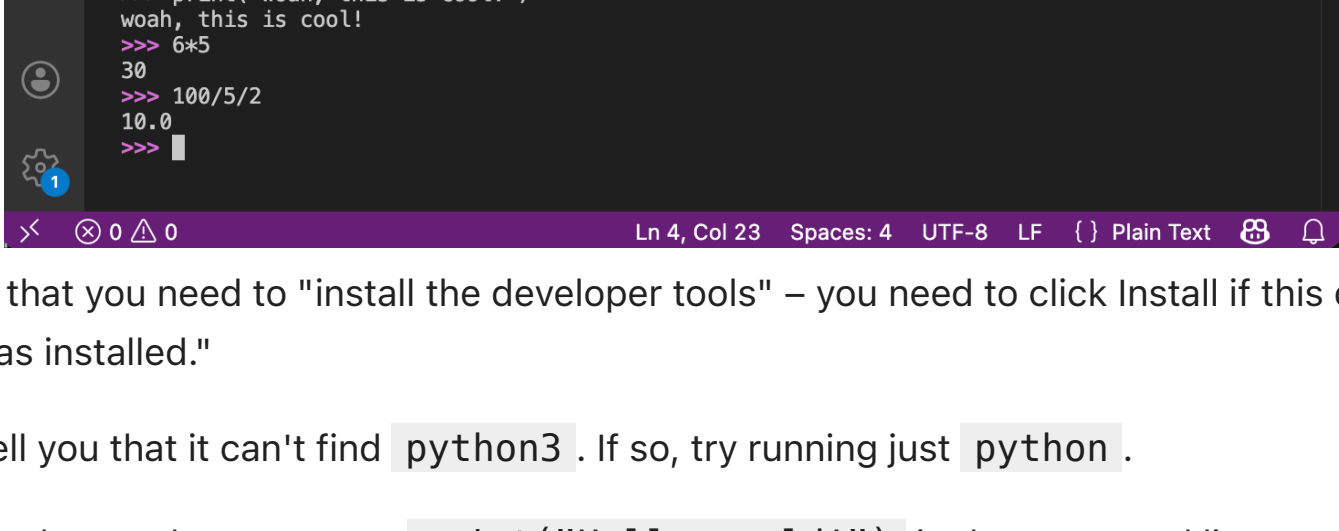
Your VSCode window should now be split between your original text file on top and the terminal below. It should look something like:



If this is your first time opening the terminal, congratulations! Let's now use it to make sure your Python installation is working.

Start by first typing the command `python3` into the terminal and hitting enter. If your installation is working, you should now have three carets preceding your blinking cursor. This allows you to directly run Python line-by-line!

Here's an example of me interacting with Python through the command line:



If you're on a Mac, it might tell you that you need to "install the developer tools" – you need to click Install if this option pops up! Let it run until your machine tells you "The software was installed."

If you're using Windows, it might tell you that it can't find `python3`. If so, try running just `python`.

Once you've successfully run the code, run the statement `print("Hello world!")` in the command line.

To exit the command line, you can type `exit()` and then hit enter.

The other way to access the terminal is through an app meant for that purpose on your machine. On Macs, this app is called `Terminal`. On Windows, getting there is slightly more complicated: follow the instructions on [this site](#). Make sure you can open the command line on your machine – from now on, you can decide if you want to use the terminal through VSCode or through the app.

Terminal Commands

There are many specific commands you can run through the command line. For this class, you'll only need to know three: `pwd`, `ls`, and `cd`. Let's go through each command step-by-step.

Return to the terminal in VSCode, where you last exited from Python. Type `pwd` and then hit enter. `pwd` stands for "print working directory." The command should print the location at which your terminal and command line are currently running in your machine's file structure. When I run the command, it does the following:

```
rebeccahicke@dhcp-v12041-32057 ~ % pwd
/Users/rebeccahicke
rebeccahicke@dhcp-v12041-32057 ~ %
```

This output is the name of the folder at which I'm located. Folders and directories are the same thing. The slashes (`/`) show subfolders; I'm in a folder called `rebeccahicke` inside of a parent folder called `Users`. If you're on a Windows machine, you may get an output of the form `C:\Users\[your_name]` instead – this is functionally the same thing! Note that Windows outputs will use backslashes instead of forward slashes.

Now, let's explore `ls`. `ls` stands for list; running it lists everything (files and (sub)folders) in your current directory. For me, running `ls` prints the following:

```
rebeccahicke@dhcp-v12041-32057 ~ % ls
Applications  Desktop  Movies  Public  Recent  Pictures  Downloads  Desktop  Library  Mail  Music  Network  Sites  Texts  Utilities
Applications  Desktop  Movies  Public  Recent  Pictures  Downloads  Desktop  Library  Mail  Music  Network  Sites  Texts  Utilities
Applications  Desktop  Movies  Public  Recent  Pictures  Downloads  Desktop  Library  Mail  Music  Network  Sites  Texts  Utilities
```

What exactly prints here will depend on what you have on your machine and what operating system you're working in. Try it out yourself!

Finally, we'll explore the `cd`, or change directory, command. As it says, this command allows you to navigate through your machine's file system by changing the directory you're in. This is probably the most important command you'll learn.

Let's try to navigate to your `info1100/labs` folder. My folder for this course is stored in Documents, so first I would write `cd Documents`. If you know the entire *path* to your subfolder, you can navigate through several folders at once. For example, from my home directory I could type `cd Documents/teaching/info1100-su26/labs` and be taken to the folder where this document was stored! (This path will look different from yours, since I have this class stored with my teaching materials!)

If you want to go back up a folder, you use two dots; that is, you can write `cd ..`.

With those skills, now try and navigate to your `labs` folder! Once there, run `pwd` again to make sure you're in the right place.

Take a screen shot of your interactions with the terminal and save the file as `lab-1-intro.png` to the same `info1100/labs` folder.

Two final tips might make interacting with the terminal easier for you:

- Tab completion:** When typing in the terminal, if you hit the `tab` key, the command line will try to fill in what you're writing. For example, if I'm in my home directory and I type `cd Doc` then hit `tab`, it will fill in `cd Documents`.
- Up and down arrows:** The up and down arrows can be used to go through the commands you've already run. Hit the up arrow once and the last command you ran will appear before your cursor, and so on.

Running Python from a File

Now, that we've introduced you to the terminal and learned how to interact with Python there, let's learn how to run Python from a file.

The good news, is that we already have a text editor open! Go back to the VSCode window where you created `lab-1-intro.txt`. First, create a new file (go to "File" then "New File") and then paste the following block of code into it:

```
print("Welcome to INFO1100!")

name = input("Enter your name: ")
print("We're going to have a great together, " + name + "!")
Then, save this new file as lab-1-intro.py to your info1100/labs folder. Be careful – you do need the .py extension! Make sure you don't put a space after the extension. When saving the file, VSCode may ask if you want to install the recommended Python extension; if it does, you should click install!
```

After you save the file, the code should *colorize* (which we often call *syntax highlighting*).

For Windows users, please make sure you're showing all file extensions! It'll make things much less complicated if you can see exactly what kind of file you're working with at all times.

Finally, let's run this file! Go back to the terminal where you've changed your directory to the `info1100/labs` folder. Run `ls` to make sure your `lab-1-intro.py` file is saved to this location. Then, run the command `python3 lab-1-intro.py`. (Again, remember that in windows this might be `python lab-1-intro.py` instead.)

The program should run! If it doesn't, feel free to try and debug it yourself, but if you get stuck, please come ask me for help!

Creating a Virtual Environment

We're now going to create something called a "virtual environment," which allows you to easily manage your version of Python and any additional packages we may want to download later this term. This step may not make a huge amount of sense yet, but it's important to have set up, and will make more sense later in the term!

To set up a virtual environment, go back to the terminal window you have open. Navigate through your folder structure using `cd` until you're in the `info1100` folder. (Make sure you're just in `info1100`, not `info1100/labs`!).

Then, run `python3 -m venv .venv`. From now on, before you run any code from the terminal, you should first run `source .venv/bin/activate` in the `info1100` folder.

Running Python from a Notebook

We'll be writing most of our assignments in `.py` files like the ones you just created. But your labs and guided reading questions (GRQs) will be done in something called an interactive notebook, which allows you to combine code blocks and text very easily.

In fact, this document itself was first written as an interactive notebook... You've probably been reading these instructions so far from the PDF version of the lab file. But now, we're going to open it as an interactive notebook, or a `.ipynb` file. To do this, first download the file named `lab-1.ipynb` from the course website and place it in your `labs` folder. Then, download the zip folder called `lab-files` to your `labs` folder and double click on the `.zip` file to open it. This folder just contains extra materials we'll use for the rest of term.

Now, we'll open your course materials as a project in VSCode. To do this, go to the "File" menu and select "Open Folder". When given the option, select your `info1100` folder to open. Now, in the lefthand of your VSCode window you should be able to see all of the folders and files in your course folder. Go to the `labs` subfolder and open the file `lab-1.ipynb`. You should be able to see exactly the content you've been working through so far! Scroll down to this spot in the assignment.

Welcome to an interactive notebook! All of the text here is written in something called *markdown* which allows me to add formatting to plain text. Double click on any of the cells in the notebook and you'll be able to see the markdown formatting which has created this document.

When we're working with guided reading questions, I'll ask you to both answer questions and to write code. Those should take place in *markdown* and *code* cells respectively. If you hover between any of the cells of text, you'll see buttons giving you the ability to add new cells that are either code or markdown. (There's also an option for AI generation – ignore this! When we use AI chatbots in this class, I'll ask you to interact with them through a chat interface like ChatGPT.)

You shouldn't usually need to worry about creating new cells; I'll mark where answers should be put in notebooks. For example, text responses will always be demarcated by two horizontal lines. Fill in the required material by replacing the *instructions* with the requested material. Let's try filling in your name and netID below as practice!

Name: [Your name here]

ID: [Your netID here]

You may also need to run code using cell blocks. An example of a code block is below. Replace the comment (`# Write your code here`) with the statement `print("Hello world!")`. Then, run the block. You can do this either by hitting the sideways triangle that appears when you hover over the cell or putting your cursor in the terminal and pressing shift + enter (or return).

The first time you run a code block, it'll ask you to select your Python version. From the dropdown menu that appears at the top of the menu click "Python Environments" and then select the option named `.venv` (which will likely be bolded and have a star next to it). Then, run your code block. When it's successfully run a green check mark should appear in the bottom left corner. If you're asked to install the `ipykernel` extension, allow the code to do so.

Conclusion

Phew, that was a lot of material! But you've done the set up that will carry us through the rest of this term. Remember that if you got stuck at any point, please reach out to me; it's essential that you've completed this set up before the course continues.

To complete this lab, please submit the `lab-1-intro.txt`, `lab-1-intro.png`, and `lab-1-intro.py` files, along with the edited version of this file (`lab-1.ipynb`). This should be the most involved lab you have to complete all term!